ESP32-I2C-MQTT

1 Communication workflow

setup() starts Serial at 115200 baud and I²C on GPIO 21 (SDA) and GPIO 22 (SCL). It initializes the ADXL345 with a ±2 g range, creates the queue, connects to Wi-Fi and MQTT, then starts both tasks. Application tasks do not start until the sensor and network connections are ready.

ADXL345 → SensorTask → SensorData queue → MqttTask
MqttTask → Wi-Fi and MQTT → Mosquitto → Subscribers

sensorTask() reads x, y and z acceleration in m/s². classifyOrientation() selects LEFT or RIGHT when |x| > |y|, according to the sign of x; otherwise it selects UP for positive y and DOWN for nonpositive y. The z value is transmitted but does not affect orientation.

mqttTask() checks connectivity, calls mqttClient.loop(), and receives a queued sample. The first sample establishes a baseline. Later samples are published only when their orientation differs from the previous dequeued sample. Messages use topic ESP32/ADXL345/ and a JSON payload formatted into a 100-byte buffer, for example:

{"orientation":"RIGHT", "x":9.70, "y":0.10, "z":0.30}

The Wokwi environment uses Wokwi-GUEST and host.wokwi.internal:1883. Real hardware reads credentials from secrets.h and uses 192.168.0.30:1883. MQTT authentication is supplied when MQTT_USER is nonempty. Docker runs Mosquitto on port 1883 with anonymous access and persistent storage; the controller uses unencrypted TCP.

2 FreeRTOS task architecture

Task	Configuration	Responsibility
SensorTask	Priority 1; stack argument 4096; no explicit core affinity	Read and classify the sensor; enqueue a sample every 500 ms using vTaskDelayUntil().
MqttTask	Priority 1; stack argument 8192; pinned to core 0	Reconnect, service MQTT and process one sample per iteration; delay 1000 ms afterward.

The sensor task owns runtime accelerometer access; the MQTT task owns runtime networking. Their queue synchronizes data transfer, so no application mutex is used for these objects. Arduino loop() only delays for one second. Task creation results are unchecked. The 500 ms acquisition period is a target: sensor access, recovery and queue blocking can cause overruns.

3 Inter task communication

```
enum class Orientation { UP, DOWN, LEFT, RIGHT };
struct SensorData {
  float x, y, z;
  Orientation orientation;
};
queue = xQueueCreate(10, sizeof(SensorData));
```

The shared QueueHandle_t identifies a FIFO queue containing up to ten SensorData records. xQueueSend() copies the complete sample into the queue, and xQueueReceive() removes the oldest sample into the consumer's local current variable. Records contain values, not pointers, so their lifetime is independent of the producer's local variable.

Both send and receive use a 500 ms timeout. A receive is processed only when its result is pdTRUE. The send result is ignored: if the queue remains full, the new sample is silently lost. prev and hasPrev are private to mqttTask() and track the last dequeued orientation; samples contain no timestamp, sequence number or validity flag.

The nominal producer rate is two samples per second, while the consumer processes roughly one per second because of its one-second delay. The queue therefore tends to fill even with a healthy connection. During network recovery it stops draining entirely, so older readings accumulate and new samples may be dropped. Ten slots provide temporary buffering, not guaranteed delivery.

4 Robustness fault detection and recovery

Condition	Implemented detection and response
Sensor initialization failure	begin() failure is logged; setup() retries every 1000 ms until the sensor is available.
Reported sensor read failure	If getEvent() returns false, log the failure, call configureAccelerometer(), skip enqueueing and wait for the next scheduled cycle.
Queue allocation failure	A null handle is logged; setup() returns without starting either application task. No retry is implemented.
Wi-Fi failure	Check WiFi.status(); connectWifi() calls WiFi.begin() and polls every 2000 ms until connected.
MQTT failure	Check connected(); retry connection every 2000 ms, log mqttClient.state(), and restore Wi-Fi when needed.

Retry delays yield execution, allowing SensorTask to run during runtime network recovery. Connection helpers have no overall timeout. Sensor read fault detection depends on what the driver reports through getEvent(); the reconfiguration result inside sensorTask() is ignored, so physical sensor recovery requires hardware validation.

Current limits: queue overflow and task creation failures are unchecked. A failed publish still updates prev, so the same orientation is not retried. There is no durable offline buffer, application acknowledgement, orientation hysteresis or unique device client ID; every controller connects as ESP32.

Priority improvements are to resolve the sampling and consumption rate mismatch, check failure results, retain failed publications for retry, and assign unique client IDs. Verify recovery by interrupting Wi-Fi, stopping the broker and disconnecting the sensor. These are proposed checks, not completed test results.